```
const RANGE_PRESETS = {
    adult: {
        male: {
            hgb: { label: 'Hemoglobin', low: 13.5, high: 17.5, unit: 'g/dL', inverseRisk: false },
            wbc: { label: 'WBC', low: 4.0, high: 11.0, unit: 'x10^3/uL', inverseRisk: false },
            plt: { label: 'Platelets', low: 150, high: 450, unit: 'x10^3/uL', inverseRisk: false },
            alt: { label: 'ALT', low: 7, high: 56, unit: 'U/L', inverseRisk: false },
            ast: { label: 'AST', low: 10, high: 40, unit: 'U/L', inverseRisk: false },
            bilirubin: { label: 'Total Bilirubin', low: 0.1, high: 1.2, unit: 'mg/dL', inverseRisk: false },
            creatinine: { label: 'Creatinine', low: 0.7, high: 1.3, unit: 'mg/dL', inverseRisk: false },
            bun: { label: 'BUN', low: 7, high: 20, unit: 'mg/dL', inverseRisk: false },
            egfr: { label: 'eGFR', low: 90, high: 140, unit: 'mL/min/1.73mÂ²', inverseRisk: true },
            tc: { label: 'Total Cholesterol', low: 120, high: 200, unit: 'mg/dL', inverseRisk: false },
            ldl: { label: 'LDL', low: 0, high: 130, unit: 'mg/dL', inverseRisk: false },
            hdl: { label: 'HDL', low: 40, high: 90, unit: 'mg/dL', inverseRisk: true },
            tg: { label: 'Triglycerides', low: 0, high: 150, unit: 'mg/dL', inverseRisk: false }
        },
        female: {
            hgb: { label: 'Hemoglobin', low: 12.0, high: 15.5, unit: 'g/dL', inverseRisk: false },
            wbc: { label: 'WBC', low: 4.0, high: 11.0, unit: 'x10^3/uL', inverseRisk: false },
            plt: { label: 'Platelets', low: 150, high: 450, unit: 'x10^3/uL', inverseRisk: false },
            alt: { label: 'ALT', low: 7, high: 56, unit: 'U/L', inverseRisk: false },
            ast: { label: 'AST', low: 10, high: 40, unit: 'U/L', inverseRisk: false },
            bilirubin: { label: 'Total Bilirubin', low: 0.1, high: 1.2, unit: 'mg/dL', inverseRisk: false },
            creatinine: { label: 'Creatinine', low: 0.6, high: 1.1, unit: 'mg/dL', inverseRisk: false },
            bun: { label: 'BUN', low: 7, high: 20, unit: 'mg/dL', inverseRisk: false },
            egfr: { label: 'eGFR', low: 90, high: 140, unit: 'mL/min/1.73mÂ²', inverseRisk: true },
            tc: { label: 'Total Cholesterol', low: 120, high: 200, unit: 'mg/dL', inverseRisk: false },
            ldl: { label: 'LDL', low: 0, high: 130, unit: 'mg/dL', inverseRisk: false },
            hdl: { label: 'HDL', low: 50, high: 100, unit: 'mg/dL', inverseRisk: true },
            tg: { label: 'Triglycerides', low: 0, high: 150, unit: 'mg/dL', inverseRisk: false }
        }
    },
    pediatric: {
        male: {
            hgb: { label: 'Hemoglobin', low: 11.5, high: 15.5, unit: 'g/dL', inverseRisk: false },
            wbc: { label: 'WBC', low: 4.5, high: 13.5, unit: 'x10^3/uL', inverseRisk: false },
            plt: { label: 'Platelets', low: 150, high: 450, unit: 'x10^3/uL', inverseRisk: false },
            alt: { label: 'ALT', low: 7, high: 45, unit: 'U/L', inverseRisk: false },
            ast: { label: 'AST', low: 10, high: 45, unit: 'U/L', inverseRisk: false },
            bilirubin: { label: 'Total Bilirubin', low: 0.1, high: 1.0, unit: 'mg/dL', inverseRisk: false },
            creatinine: { label: 'Creatinine', low: 0.4, high: 1.0, unit: 'mg/dL', inverseRisk: false },
            bun: { label: 'BUN', low: 5, high: 18, unit: 'mg/dL', inverseRisk: false },
            egfr: { label: 'eGFR', low: 90, high: 150, unit: 'mL/min/1.73mÂ²', inverseRisk: true },
```

```
    tc: { label: 'Total Cholesterol', low: 120, high: 170, unit: 'mg/dL', inverseRisk: false },
    ldl: { label: 'LDL', low: 0, high: 110, unit: 'mg/dL', inverseRisk: false },
    hdl: { label: 'HDL', low: 40, high: 90, unit: 'mg/dL', inverseRisk: true },
    tg: { label: 'Triglycerides', low: 0, high: 130, unit: 'mg/dL', inverseRisk: false }
  },
  female: {
    hgb: { label: 'Hemoglobin', low: 11.5, high: 15.5, unit: 'g/dL', inverseRisk: false },
    wbc: { label: 'WBC', low: 4.5, high: 13.5, unit: 'x10^3/uL', inverseRisk: false },
    plt: { label: 'Platelets', low: 150, high: 450, unit: 'x10^3/uL', inverseRisk: false },
    alt: { label: 'ALT', low: 7, high: 45, unit: 'U/L', inverseRisk: false },
    ast: { label: 'AST', low: 10, high: 45, unit: 'U/L', inverseRisk: false },
    bilirubin: { label: 'Total Bilirubin', low: 0.1, high: 1.0, unit: 'mg/dL', inverseRisk: false },
    creatinine: { label: 'Creatinine', low: 0.4, high: 1.0, unit: 'mg/dL', inverseRisk: false },
    bun: { label: 'BUN', low: 5, high: 18, unit: 'mg/dL', inverseRisk: false },
    egfr: { label: 'eGFR', low: 90, high: 150, unit: 'mL/min/1.73mÂ²', inverseRisk: true },
    tc: { label: 'Total Cholesterol', low: 120, high: 170, unit: 'mg/dL', inverseRisk: false },
    ldl: { label: 'LDL', low: 0, high: 110, unit: 'mg/dL', inverseRisk: false },
    hdl: { label: 'HDL', low: 45, high: 95, unit: 'mg/dL', inverseRisk: true },
    tg: { label: 'Triglycerides', low: 0, high: 130, unit: 'mg/dL', inverseRisk: false }
  }
};

const STORAGE_KEY_RANGES = 'lab-sim-ranges-v2';
const STORAGE_KEY_HISTORY = 'lab-sim-history-v1';
const STORAGE_KEY_PROFILE = 'lab-sim-profile-v1';

const form = document.getElementById('lab-form');
const rangeEditor = document.getElementById('range-editor');
const parameterResults = document.getElementById('parameter-results');
const patternNotes = document.getElementById('pattern-notes');
const overallStatus = document.getElementById('overall-status');
const overallNote = document.getElementById('overall-note');
const lastAnalyzed = document.getElementById('last-analyzed');

const ageGroupSelect = document.getElementById('age-group');
const sexSelect = document.getElementById('sex');
const applyPresetBtn = document.getElementById('apply-preset');

const saveRangesBtn = document.getElementById('save-ranges');
const restoreDefaultsBtn = document.getElementById('restore-defaults');
const demoBtn = document.getElementById('load-demo');
const resetBtn = document.getElementById('reset-form');
const exportCsvBtn = document.getElementById('export-csv');
```

```
const exportPdfBtn = document.getElementById('export-pdf');

const trendMetricSelect = document.getElementById('trend-metric');
const clearHistoryBtn = document.getElementById('clear-history');
const trendCanvas = document.getElementById('trend-canvas');
const historyTableBody = document.querySelector('#history-table tbody');

let currentProfile = loadProfile();
let ranges = loadRanges();
let history = loadHistory();
let lastAnalysis = null;

syncProfileSelectors();
populateTrendMetricOptions();
renderRangeEditor();
renderHistoryTable();
drawTrendChart();

function getPreset(ageGroup, sex){
    return structuredClone(RANGE_PRESETS[ageGroup][sex]);
}

function loadProfile(){
    const raw = localStorage.getItem(STORAGE_KEY_PROFILE);
    if (!raw) return { ageGroup: 'adult', sex: 'female' };
    try{
        const parsed = JSON.parse(raw);
        if (RANGE_PRESETS[parsed.ageGroup] && RANGE_PRESETS[parsed.ageGroup][parsed.sex]){
            return parsed;
        }
    } catch{
        // ignore malformed profile
    }
    return { ageGroup: 'adult', sex: 'female' };
}

function saveProfile(){
    localStorage.setItem(STORAGE_KEY_PROFILE, JSON.stringify(currentProfile));
}

function syncProfileSelectors(){
    ageGroupSelect.value = currentProfile.ageGroup;
    sexSelect.value = currentProfile.sex;
```

```
}

function loadRanges() {
    const fallback = getPreset(currentProfile.ageGroup, currentProfile.sex);
    const raw = localStorage.getItem(STORAGE_KEY_RANGES);
    if (!raw) return fallback;

    try {
        const parsed = JSON.parse(raw);
        const merged = structuredClone(fallback);
        for (const key of Object.keys(merged)) {
            if (parsed[key] && Number.isFinite(parsed[key].low) && Number.isFinite(parsed[key].high)) {
                merged[key].low = Number(parsed[key].low);
                merged[key].high = Number(parsed[key].high);
            }
        }
        return merged;
    } catch {
        return fallback;
    }
}

function saveRanges() {
    localStorage.setItem(STORAGE_KEY_RANGES, JSON.stringify(ranges));
}

function loadHistory() {
    const raw = localStorage.getItem(STORAGE_KEY_HISTORY);
    if (!raw) return [];
    try {
        const parsed = JSON.parse(raw);
        if (Array.isArray(parsed)) return parsed;
    } catch {
        // ignore malformed history
    }
    return [];
}

function saveHistory() {
    localStorage.setItem(STORAGE_KEY_HISTORY, JSON.stringify(history));
}

function renderRangeEditor() {
```

```
    rangeEditor.innerHTML = '';
    for (const key of Object.keys(ranges)) {
        const row = document.createElement('div');
        row.className = 'range-row';
        row.innerHTML = `
            <strong>${ranges[key].label} (${ranges[key].unit})</strong>
            <input type="number" step="0.1" data-key="${key}" data-side="low"
value="${ranges[key].low}" />
            <input type="number" step="0.1" data-key="${key}" data-side="high"
value="${ranges[key].high}" />
        `;
        rangeEditor.appendChild(row);
    }
}

function populateTrendMetricOptions() {
    trendMetricSelect.innerHTML = '';
    const sample = getPreset('adult', 'female');
    for (const key of Object.keys(sample)) {
        const opt = document.createElement('option');
        opt.value = key;
        opt.textContent = `${sample[key].label} (${sample[key].unit})`;
        trendMetricSelect.appendChild(opt);
    }
}

function classifyValue(value, low, high, inverseRisk = false) {
    if (!Number.isFinite(value)) return { status: 'missing', severity: 0, detail: 'No value' };

    const under = value < low;
    const over = value > high;

    if (!under && !over) {
        return { status: 'normal', severity: 0, detail: `Within range (${low} – ${high})` };
    }

    const deviation = under ? (low - value) / Math.max(Math.abs(low), 1e-6) : (value - high) /
Math.max(Math.abs(high), 1e-6);
    const critical = deviation >= 0.5;

    if (under) {
        const highRiskLow = inverseRisk;
        if (critical || highRiskLow) {
            return {
```

```
            status: 'critical',
            severity: highRiskLow ? 3 : 2,
            detail: `Below range (${low} - ${high})`
        };
    }
    return { status: 'low', severity: 1, detail: `Below range (${low} - ${high})` };
}

if (critical && !inverseRisk) {
    return { status: 'critical', severity: 2, detail: `Above range (${low} - ${high})` };
  }

  return { status: 'high', severity: 1, detail: `Above range (${low} - ${high})` };
}

function computeOverall(perParam){
  const maxSeverity = perParam.reduce((max, p) => Math.max(max, p.classification.severity), 0);
  const abnormalCount = perParam.filter((p) => !['normal', 'missing'].includes(p.classification.status)).length;

  if (maxSeverity >= 3) {
    return {
      className: 'critical',
      label: 'Urgent clinical review',
      note: 'At least one value is significantly concerning. Seek urgent professional evaluation.',
      abnormalCount
    };
  }

  if (maxSeverity >= 2 || abnormalCount >= 4) {
    return {
      className: 'risk',
      label: 'Needs doctor review',
      note: 'Multiple or severe abnormalities detected. Prompt clinician review is advised.',
      abnormalCount
    };
}

if (abnormalCount > 0) {
  return {
    className: 'warn',
    label: 'Borderline / follow-up advised',
```

```
        note: 'Some values are outside reference ranges. Discuss with a healthcare professional.',
        abnormalCount
      };
    }

    return{
      className:'safe',
      label:'Within configured safe range',
      note: 'All entered values are within configured ranges. This is not a diagnosis.',
      abnormalCount
    };
}

function generatePatternNotes(perParam){
    const statusByKey = Object.fromEntries(perParam.map((p) => [p.key, p.classification.status]));
    const notes = [];

    if (statusByKey.hgb === 'low' || statusByKey.hgb === 'critical') {
      notes.push('Low hemoglobin may align with anemia patterns. Consider correlation with iron/B12 studies and symptoms.');
    }

    if (['high', 'critical'].includes(statusByKey.wbc)) {
      notes.push('Raised WBC can occur in infection/inflammation patterns and should be interpreted clinically.');
    }

    if (['low', 'critical'].includes(statusByKey.plt)){
      notes.push('Low platelets may increase bleeding risk patterns. Clinical confirmation is important.');
    }

    if (['high', 'critical'].includes(statusByKey.alt) && ['high', 'critical'].includes(statusByKey.ast))
{
      notes.push('Concurrent ALT/AST elevation may suggest hepatocellular injury patterns.');
    }

    if (['high', 'critical'].includes(statusByKey.bilirubin)) {
      notes.push('Elevated bilirubin can align with cholestatic/hemolytic/liver dysfunction patterns depending on context.');
    }
```

```
  if (['high', 'critical'].includes(statusByKey.creatinine) || ['high', 'critical'].includes(statusByKey.bun) || ['critical', 'low'].includes(statusByKey.egfr)) {
    notes.push('Kidney function abnormalities may align with renal impairment patterns; trend over time matters.');
  }

  if (['high', 'critical'].includes(statusByKey.ldl) || ['high', 'critical'].includes(statusByKey.tc) || ['high', 'critical'].includes(statusByKey.tg) || ['critical', 'low'].includes(statusByKey.hdl)) {
    notes.push('Lipid abnormalities may indicate increased cardiovascular risk patterns. Lifestyle + clinician review recommended.');
  }

  if (notes.length === 0) {
    notes.push('No obvious pattern flags from entered values. Continue routine preventive care as advised by your clinician.');
  }

  notes.push('Educational note only: this does not provide diagnosis or treatment decisions.');
  return notes;
}

function renderResults(analysis) {
  parameterResults.innerHTML = '';
  for (const item of analysis.perParam) {
    const div = document.createElement('div');
    div.className = 'result-item';
    const badgeClass = item.classification.status === 'normal'
      ? 'normal'
      : item.classification.status === 'critical'
        ? 'critical'
        : item.classification.status;

    div.innerHTML = `
      <div>
        <span class="name">${item.label}</span>
        <span class="badge ${badgeClass}">${item.classification.status.toUpperCase()}</span>
      </div>
      <div>Entered: ${Number.isFinite(item.value) ? item.value : '-'} ${item.unit}</div>
      <div class="muted">${item.classification.detail}</div>
    `;
    parameterResults.appendChild(div);
```

```
    }

    overallStatus.className = `pill ${analysis.overall.className}`;
    overallStatus.textContent = analysis.overall.label;
    overallNote.textContent = analysis.overall.note;
    lastAnalyzed.textContent = `Last analyzed: ${new Date(analysis.timestamp).toLocaleString()}`;

    patternNotes.innerHTML = '';
    for (const note of analysis.notes) {
      const li = document.createElement('li');
      li.textContent = note;
      patternNotes.appendChild(li);
    }
  }

  function parseInputValue(key) {
    const raw = form.elements[key].value;
    if (raw === '') return NaN;
    const value = Number(raw);
    return Number.isFinite(value) ? value : NaN;
  }

  function runAnalysis() {
    const values = {};
    const perParam = [];

    for (const key of Object.keys(ranges)) {
      values[key] = parseInputValue(key);
      const r = ranges[key];
      const classification = classifyValue(values[key], r.low, r.high, r.inverseRisk);

      perParam.push({
        key,
        label: r.label,
        unit: r.unit,
        value: values[key],
        classification
      });
    }

    const overall = computeOverall(perParam);
    const notes = generatePatternNotes(perParam);
```

```
    return {
        timestamp: Date.now(),
        profile: structuredClone(currentProfile),
        values,
        perParam,
        overall,
        notes
    };
}

function pushHistory(analysis) {
    history.push({
        timestamp: analysis.timestamp,
        profile: analysis.profile,
        overallClass: analysis.overall.className,
        overallLabel: analysis.overall.label,
        abnormalCount: analysis.overall.abnormalCount,
        values: analysis.values
    });

    if (history.length > 100) {
        history = history.slice(history.length - 100);
    }

    saveHistory();
    renderHistoryTable();
    drawTrendChart();
}

function renderHistoryTable() {
    historyTableBody.innerHTML = '';

    if (history.length === 0) {
        const row = document.createElement('tr');
        row.innerHTML = `<td colspan="4" class="table-empty">No trend history yet. Run analysis to create records.</td>`;
        historyTableBody.appendChild(row);
        return;
    }

    const recent = [...history].reverse().slice(0, 12);
    for (const record of recent) {
        const row = document.createElement('tr');
        const snapshot = Object.entries(record.values)
```

```
            .slice(0,5)
            .map(([k,v]) => `${k}:${Number.isFinite(v)?v:'-'}`)
            .join('Â· ');

        row.innerHTML = `
            <td>${new Date(record.timestamp).toLocaleString()}</td>
            <td><span                                                    class="status-tag
${record.overallClass}">${record.overallLabel}</span></td>
            <td>${record.abnormalCount}</td>
            <td class="snapshot">${snapshot}${Object.keys(record.values).length > 5 ? ' ...' :
"}</td>
        `;
        historyTableBody.appendChild(row);
    }
}

function drawTrendChart(){
    const ctx = trendCanvas.getContext('2d');
    const width = trendCanvas.width;
    const height = trendCanvas.height;
    const metric = trendMetricSelect.value || 'hgb';

    ctx.clearRect(0, 0, width, height);
    ctx.fillStyle = '#fcfdff';
    ctx.fillRect(0, 0, width, height);

    const pad = { top: 20, right: 20, bottom: 35, left: 45 };
    const plotW = width - pad.left - pad.right;
    const plotH = height - pad.top - pad.bottom;

    ctx.strokeStyle = '#dfe5ef';
    ctx.lineWidth = 1;
    for (let i = 0; i <= 4; i += 1) {
        const y = pad.top + (plotH * i) / 4;
        ctx.beginPath();
        ctx.moveTo(pad.left, y);
        ctx.lineTo(width - pad.right, y);
        ctx.stroke();
    }

    ctx.beginPath();
    ctx.moveTo(pad.left, pad.top);
    ctx.lineTo(pad.left, height - pad.bottom);
    ctx.lineTo(width - pad.right, height - pad.bottom);
```

```
ctx.strokeStyle = '#9fb0cc';
ctx.stroke();

const points = history
    .map((record) => ({ ts: record.timestamp, value: record.values[metric] }))
    .filter((p) => Number.isFinite(p.value));

const label = ranges[metric]?.label ?? metric;

if (points.length < 2) {
    ctx.fillStyle = '#5b6478';
    ctx.font = '14px Inter, Arial';
    ctx.fillText(`Need at least 2 data points for ${label} trend.`, pad.left, pad.top + 20);
    return;
}

const min = Math.min(...points.map((p) => p.value));
const max = Math.max(...points.map((p) => p.value));
const spread = max - min || 1;

ctx.strokeStyle = '#2d6cdf';
ctx.lineWidth = 2;
ctx.beginPath();

points.forEach((p, index) => {
    const x = pad.left + (plotW * index) / (points.length - 1);
    const y = pad.top + ((max - p.value) / spread) * plotH;
    if (index === 0) ctx.moveTo(x, y);
    else ctx.lineTo(x, y);
});
ctx.stroke();

ctx.fillStyle = '#2d6cdf';
points.forEach((p, index) => {
    const x = pad.left + (plotW * index) / (points.length - 1);
    const y = pad.top + ((max - p.value) / spread) * plotH;
    ctx.beginPath();
    ctx.arc(x, y, 3, 0, Math.PI * 2);
    ctx.fill();
});

ctx.fillStyle = '#1d2433';
ctx.font = '12px Inter, Arial';
ctx.fillText(`${label} (${ranges[metric]?.unit ?? ''})`, pad.left, 14);
```

```
    ctx.fillText(`Min: ${min.toFixed(2)}`, width - 170, 14);
    ctx.fillText(`Max: ${max.toFixed(2)}`, width - 90, 14);
}

function toCsvRow(values) {
    return values.map((v) => {
        const s = `${v ?? ''}`;
        if (s.includes(',') || s.includes('"') || s.includes('\n')) {
            return `"${s.replaceAll('"', '""')}"`;
        }
        return s;
    }).join(',');
}

function exportCsv(analysis) {
    const lines = [];
    lines.push(toCsvRow(['Lab Safety Range Simulator Report']));
    lines.push(toCsvRow(['Timestamp', new Date(analysis.timestamp).toLocaleString()]));
    lines.push(toCsvRow(['Profile', `${analysis.profile.ageGroup} / ${analysis.profile.sex}`]));
    lines.push(toCsvRow(['Overall Status', analysis.overall.label]));
    lines.push(toCsvRow(['Overall Note', analysis.overall.note]));
    lines.push('');
    lines.push(toCsvRow(['Parameter', 'Value', 'Unit', 'Range Low', 'Range High', 'Status',
'Detail']));

    for (const item of analysis.perParam) {
        const r = ranges[item.key];
        lines.push(toCsvRow([
            item.label,
            Number.isFinite(item.value) ? item.value : '',
            item.unit,
            r.low,
            r.high,
            item.classification.status,
            item.classification.detail
        ]));
    }

    lines.push('');
    lines.push(toCsvRow(['Educational Notes']));
    for (const note of analysis.notes) {
        lines.push(toCsvRow([note]));
    }
```

```
    const blob = new Blob([lines.join('\n')], { type: 'text/csv;charset=utf-8;' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `lab-report-${new Date(analysis.timestamp).toISOString().replace(/[:.]/g,
'-')}.csv`;
    document.body.appendChild(a);
    a.click();
    document.body.removeChild(a);
    URL.revokeObjectURL(url);
}

function exportPdf(analysis) {
    const rowsHtml = analysis.perParam.map((item) => {
        const r = ranges[item.key];
        const valueText = Number.isFinite(item.value) ? item.value : '-';
        return `
            <tr>
                <td>${item.label}</td>
                <td>${valueText}${item.unit}</td>
                <td>${r.low} - ${r.high}</td>
                <td>${item.classification.status.toUpperCase()}</td>
                <td>${item.classification.detail}</td>
            </tr>
        `;
    }).join('');

    const notesHtml = analysis.notes.map((note) => `<li>${note}</li>`).join('');

    const html = `
        <!doctype html>
        <html>
            <head>
                <meta charset="UTF-8" />
                <title>Lab Report</title>
                <style>
                    body { font-family: Arial, sans-serif; margin: 24px; color: #222; }
                    h1, h2, h3 { margin-bottom: 8px; }
                    table { width: 100%; border-collapse: collapse; margin-top: 12px; }
                    th, td { border: 1px solid #ccc; padding: 8px; text-align: left; font-size: 12px; }
                    .muted { color: #666; font-size: 12px; }
                    .pill { display:inline-block; padding: 4px 8px; border-radius: 999px; background:
#eceff1; }
                    ul { margin-top: 6px; }
```

```
      </style>
    </head>
    <body>
      <h1>Lab Safety Range Simulator Report</h1>
      <p class="muted">Educational only. Not a diagnosis or treatment recommendation.</p>
      <p><strong>Timestamp:</strong> ${new Date(analysis.timestamp).toLocaleString()}</p>
      <p><strong>Profile:</strong> ${analysis.profile.ageGroup} / ${analysis.profile.sex}</p>
      <p><strong>Overall:</strong> <span class="pill">${analysis.overall.label}</span></p>
      <p>${analysis.overall.note}</p>

      <h3>Per-Parameter Interpretation</h3>
      <table>
        <thead>
          <tr>
            <th>Parameter</th>
            <th>Value</th>
            <th>Reference Range</th>
            <th>Status</th>
            <th>Detail</th>
          </tr>
        </thead>
        <tbody>${rowsHtml}</tbody>
      </table>

      <h3>Educational Pattern Notes</h3>
      <ul>${notesHtml}</ul>
    </body>
  </html>
`;

const reportWindow = window.open('', '_blank', 'width=960,height=720');
if (!reportWindow) {
  alert('Pop-up blocked. Please allow pop-ups to export PDF.');
  return;
}

reportWindow.document.open();
reportWindow.document.write(html);
reportWindow.document.close();
reportWindow.focus();
```

```
    reportWindow.print();
}

function applyProfilePreset(ageGroup, sex) {
    currentProfile = { ageGroup, sex };
    saveProfile();
    ranges = getPreset(ageGroup, sex);
    saveRanges();
    renderRangeEditor();
}

form.addEventListener('submit', (event) => {
    event.preventDefault();

    lastAnalysis = runAnalysis();
    renderResults(lastAnalysis);
    pushHistory(lastAnalysis);
    exportCsvBtn.disabled = false;
    exportPdfBtn.disabled = false;
});

saveRangesBtn.addEventListener('click', () => {
const draft = structuredClone(ranges);

for (const input of rangeEditor.querySelectorAll('input')) {
    const key = input.dataset.key;
    const side = input.dataset.side;
    const value = Number(input.value);
    if (!Number.isFinite(value)) {
        alert(`Invalid range value for ${draft[key]?.label ?? key}`);
        return;
    }
    draft[key][side] = value;
}

for (const key of Object.keys(draft)) {
    if (draft[key].low >= draft[key].high) {
        alert(`Low must be less than high for ${draft[key].label}`);
        return;
    }
}

ranges = draft;
saveRanges();
```

```
    drawTrendChart();
    alert('Reference ranges saved.');
});

restoreDefaultsBtn.addEventListener('click', () => {
    ranges = getPreset(currentProfile.ageGroup, currentProfile.sex);
    saveRanges();
    renderRangeEditor();
    drawTrendChart();
    alert('Default preset ranges restored for current age/sex profile.');
});

applyPresetBtn.addEventListener('click', () => {
    applyProfilePreset(ageGroupSelect.value,sexSelect.value);
    alert(`Applied preset: ${ageGroupSelect.value} / ${sexSelect.value}`);
});

demoBtn.addEventListener('click', () => {
    const demo = {
        hgb:10.2,
        wbc:13.6,
        plt:140,
        alt:78,
        ast:62,
        bilirubin:1.8,
        creatinine:1.9,
        bun:34,
        egfr:48,
        tc:228,
        ldl:152,
        hdl:34,
        tg:215
};

for (const [key,value] of Object.entries(demo)) {
    if (form.elements[key]) form.elements[key].value = value;
    }
});

resetBtn.addEventListener('click', () => {
    form.reset();
    parameterResults.innerHTML = '';
    patternNotes.innerHTML = '';
    overallStatus.className = 'pill neutral';
```

```
    overallStatus.textContent = 'No analysis yet';
    overallNote.textContent = 'Enter values and click Analyze.';
    lastAnalyzed.textContent = 'Last analyzed: -';
    lastAnalysis = null;
    exportCsvBtn.disabled = true;
    exportPdfBtn.disabled = true;
});

exportCsvBtn.addEventListener('click', () => {
    if (!lastAnalysis) {
        alert('Run analysis first, then export CSV.');
        return;
    }
    exportCsv(lastAnalysis);
});

exportPdfBtn.addEventListener('click', () => {
    if (!lastAnalysis) {
        alert('Run analysis first, then export PDF.');
        return;
    }
    exportPdf(lastAnalysis);
});

trendMetricSelect.addEventListener('change', () => {
    drawTrendChart();
});

clearHistoryBtn.addEventListener('click', () => {
    const ok = window.confirm('Clear all locally saved trend history?');
    if (!ok) return;
    history = [];
    saveHistory();
    renderHistoryTable();
    drawTrendChart();
});

// Initial export buttons disabled until first analysis.
exportCsvBtn.disabled = true;
exportPdfBtn.disabled = true;
```